

Analysis on Route Planning Algorithms

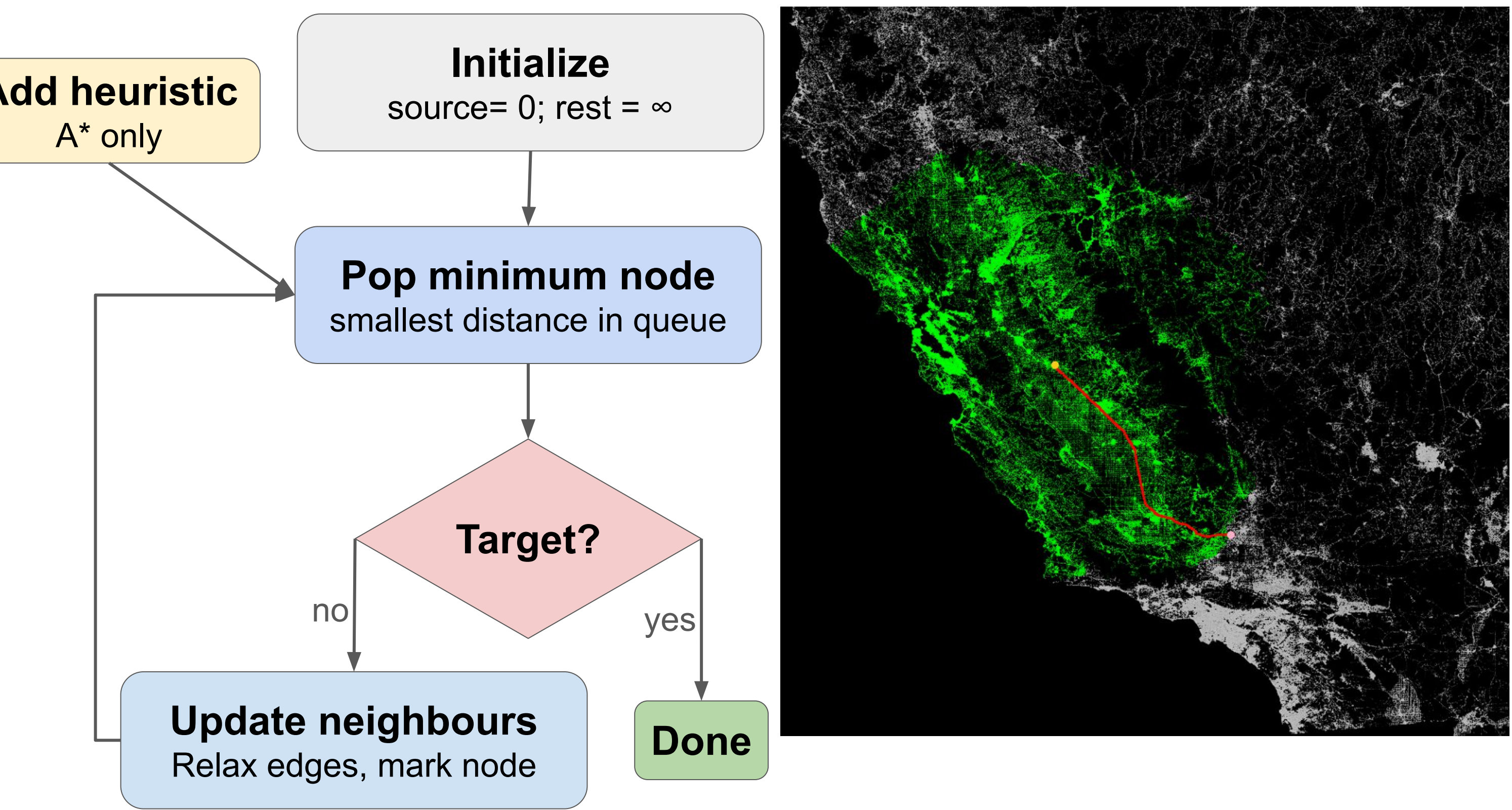
Yousif Abbas, Prof. Dr. Jan Van Den Bussche

INTRODUCTION

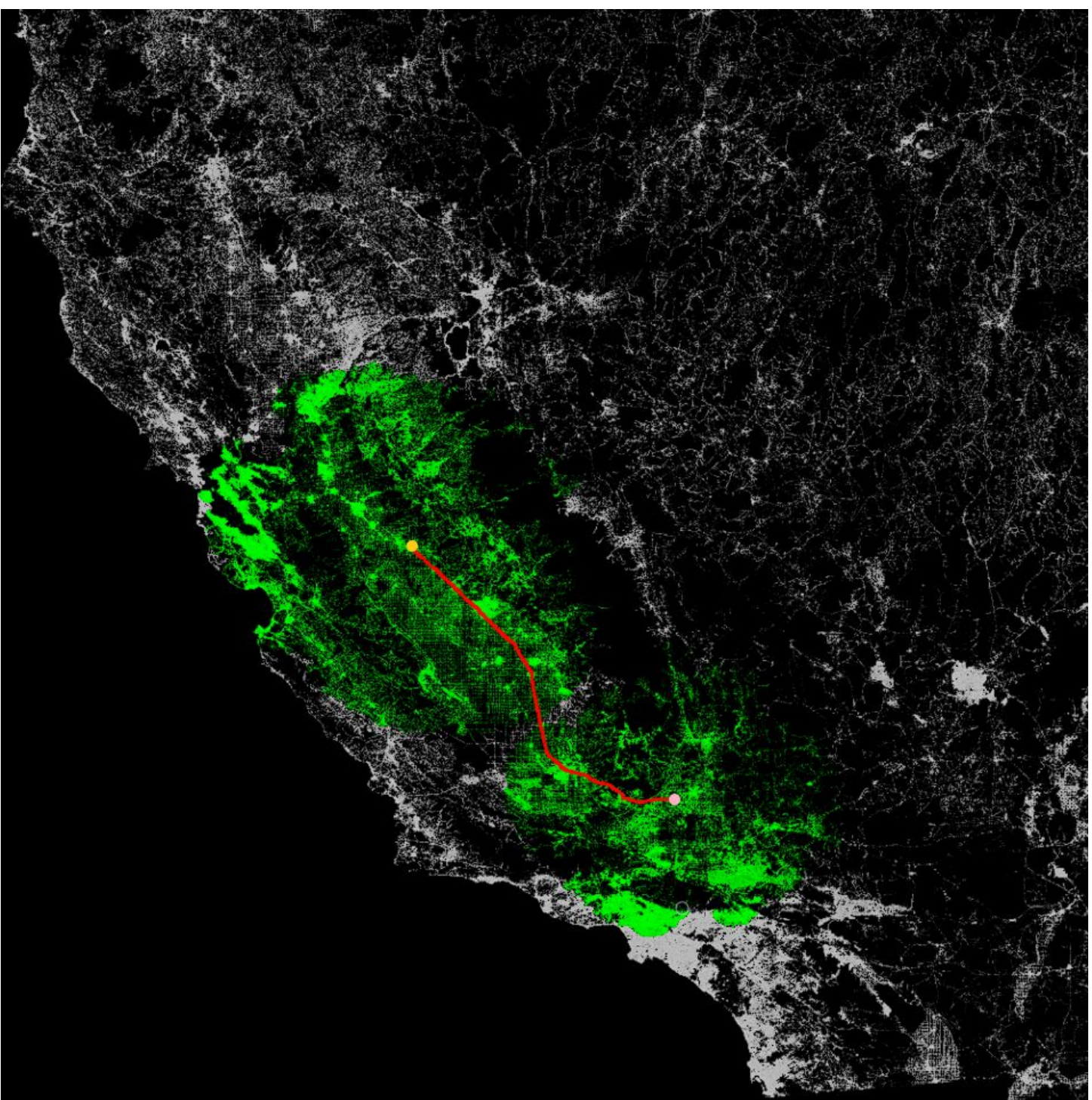
Route planning algorithms are fundamental for many applications, such as the **GPS** or **communication networks**. The performance of these algorithms is also heavily influenced by the underlying **priority queue architecture** used to manage the "frontier" of explored nodes. This study benchmarks:

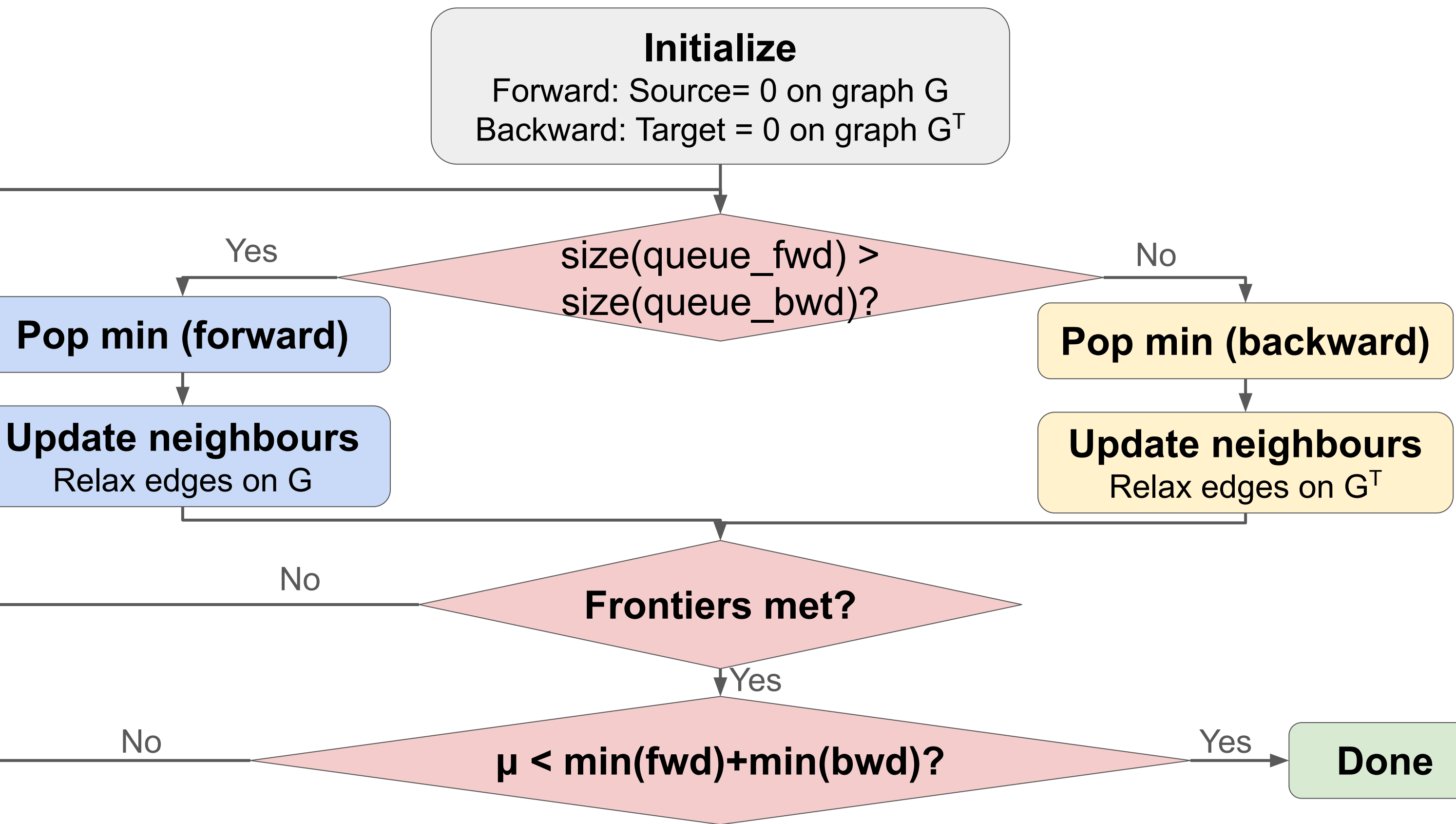
- Dijkstra
- A*
- Bidirectional variants
- Binary Min-Heap
- Fibonacci Heap
- Pairing Heap

UNIDIRECTIONAL SSSP-ALGORITHMS



BIDIRECTIONAL VARIANTS

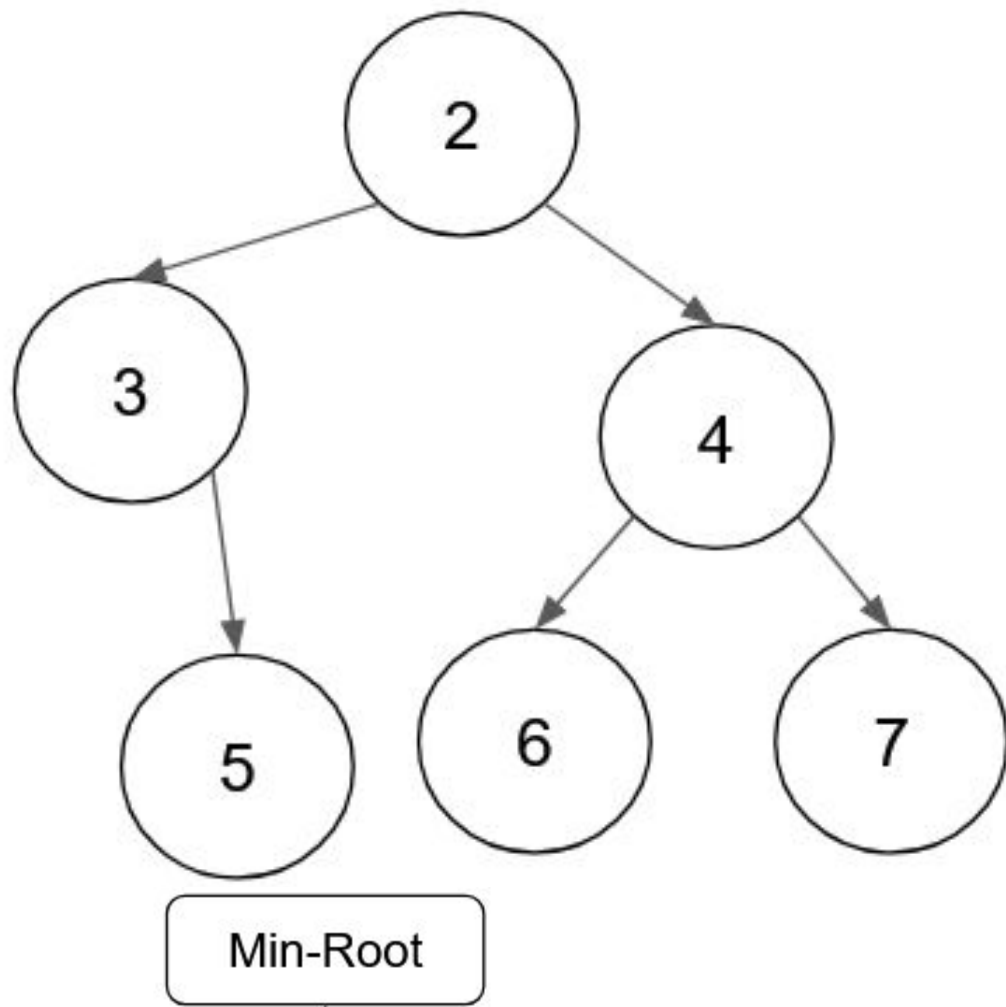
- **Concept:** Two concurrent searches meeting in the middle.
 - **Optimization:** Search space complexity drops exponentially.
 - **Adjusted Heuristics:** bidirectional A* requires feasible, consistent heuristics.
 - **Drawbacks:** Dual priority queue overhead, constant termination checks and adjusted heuristic calculation can make it slower than unidirectional search in practice.
- 



PRIORITY QUEUE ARCHITECTURES

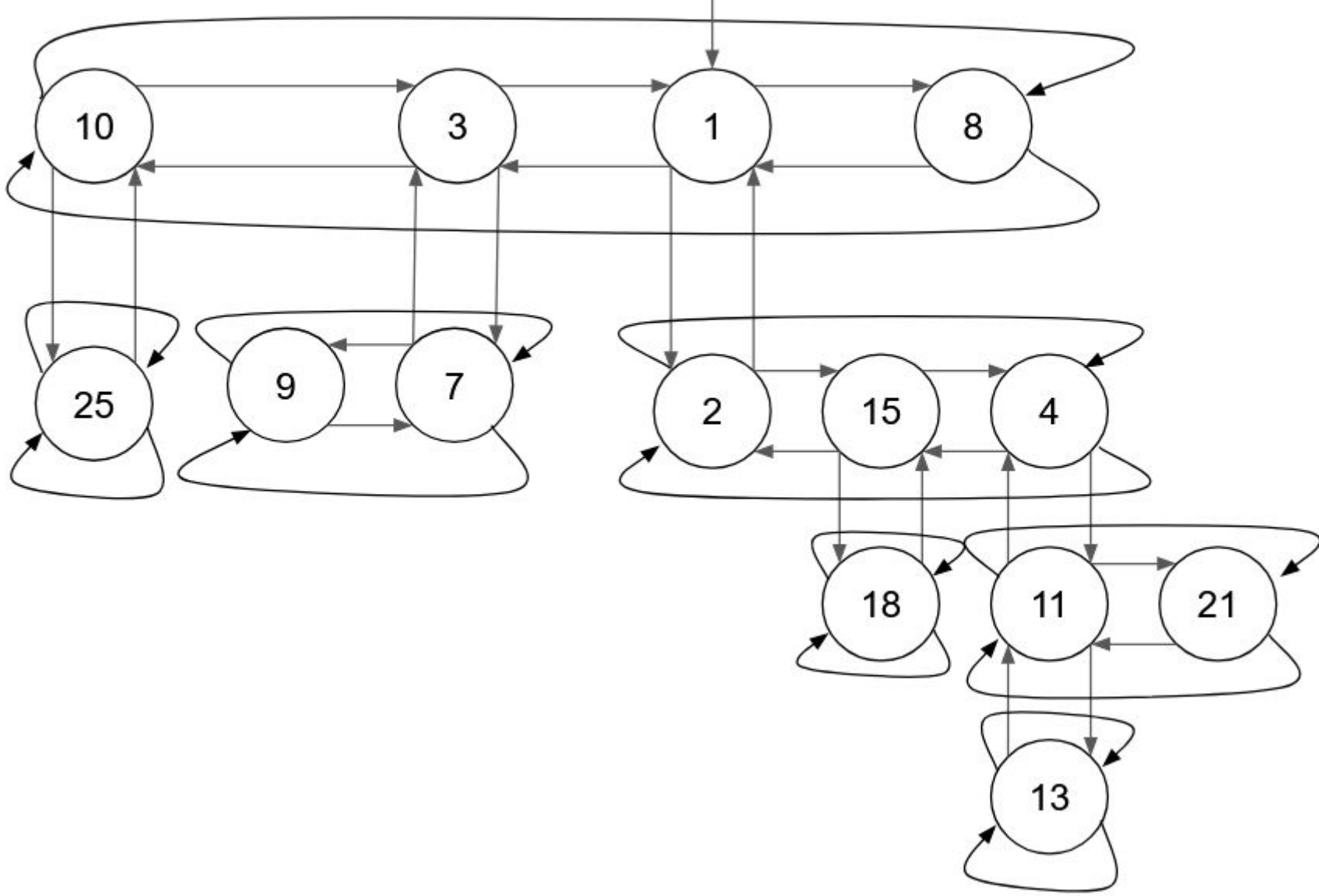
Binary Min-Heap

- **Node:** Key and value.
- **Structure:** Complete binary tree using a contiguous array.



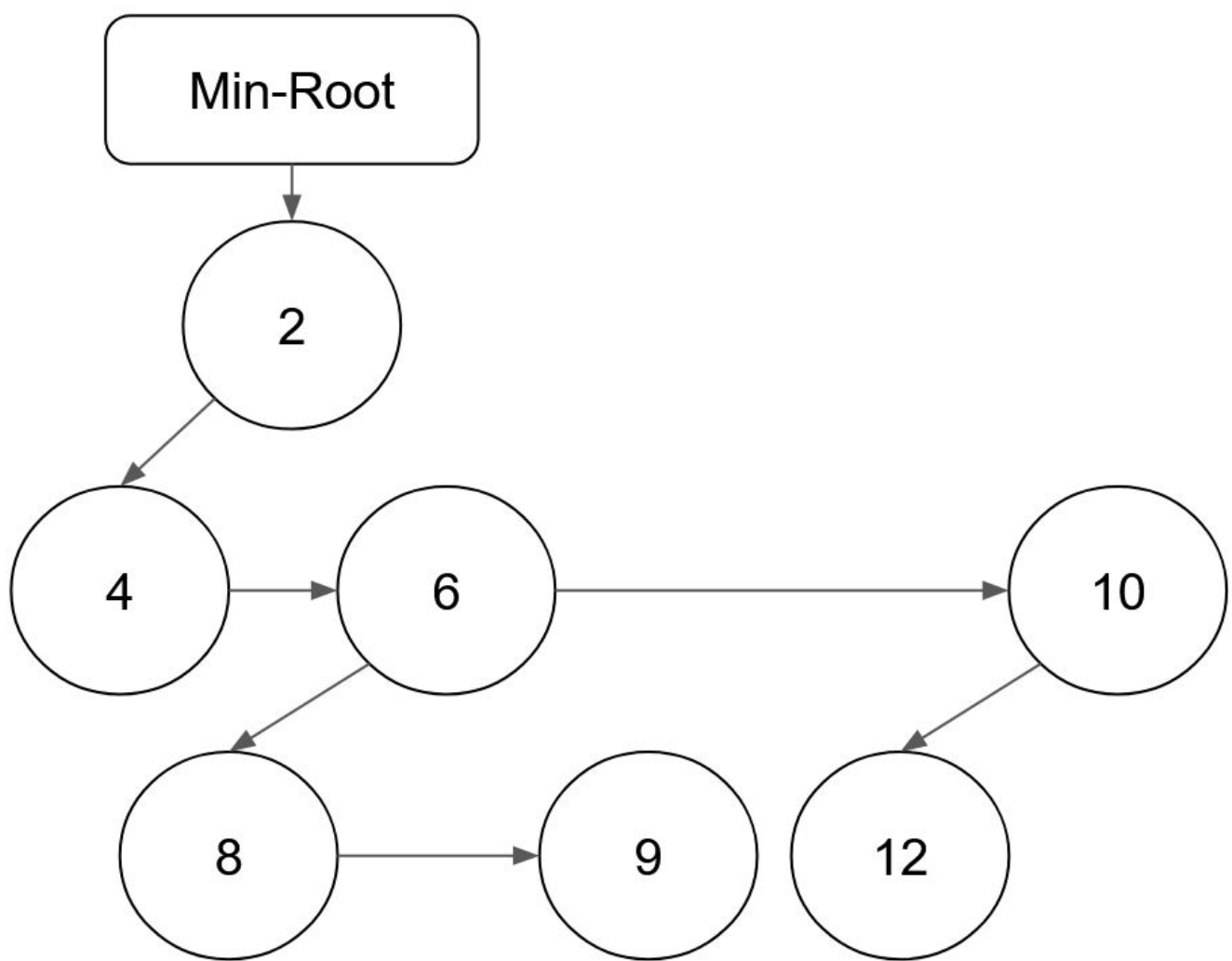
Fibonacci Heap

- **Node:** Key, value, mark, degree, parent pointer, child pointer and sibling pointers.
- **Structure:** Collection of heap-ordered trees. Roots are connected in a circular, doubly linked list.



Pairing Heap

- **Node:** Key, value, child pointer and siblings pointer. Left pointer doubles as parent pointer.
- **Structure:** Single multi-way heap-ordered tree.



	Insert	Decrease-Key	Extract-Min
Binary Min-Heap	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$
Fibonacci Heap	$O(1)^*$	$O(1)^*$	$O(\log(n))^*$
Pairing Heap	$O(1)^*$	$O(1)^{**}$	$O(\log(n))^*$

*: Amortized

**: Conjectured

RESULTS

Search Efficiency

	California		New York	
Algorithm	Time (s)	Visit %	Time (s)	Visit %
Dijkstra	0.182	51.54	0.024	49.09
Bi-Dijkstra	0.208	40.30	0.020	32.50
A*	0.100	14.22	0.007	12.31
Bi-A*	0.157	12.78	0.017	8.75

Architecture Efficiency

Data Structure	California Time (s)	Random graph Time (s)
Binary Min-Heap	0.182	0.040
Fibonacci Heap	0.356	0.040
Pairing Heap	0.236	0.041

CONCLUSION

- Bidirectional variants offer better search space but have a **longer execution time**.
- Complex heap architectures that theoretically improve the complexity of Dijkstra might **not always be better in practice**.
- A* combined with a Binary Min-Heap offers the **best balance** between execution time and search space.